CP12 Proposta de upgrade do projeto N1

Internet das Coisas | Aula 12 — Hardware, firmware-base e contratos

Prof. Edson Vaz Lopes

Identificação

Equipe (n e nome):	**Projeto (N1 - N3)**:
IOT_Irrigation Ana Julia Castelo Branco Graziela Torres Henrique Xavier Klappoth Marina Rosa Oliveira Rodrigo Xavier Klappoth Sophia Eggert	Irrigação
Integrantes presentes:	**Data / Versão:**
Ana Julia Castelo Branco Graziela Torres Marina Rosa Oliveira Sophia Eggert	27/10/2026 — v2.0

Orientações

Este checkpoint é a proposta de upgrade do projeto quo a equipe do findou na N1 para o produto da N3: cada seção conforme o plano da equipe contra o piso R1—R18 definido os `requisitos projeto N3. pdf` (documento à parte, leitura obrigatória antes de preencher este checkpoint).

Da Aula 12 até a N2 (03/11), a Unidade 2 é 100% desenho: nào hă bancada, nào hă circuito montado, nào hă firmwaro rodando no ESP32. O que se produz é documentação, cronograma, fluxogramas, diagrama de blocos e o ropositório GitHub organizado. A Unidade 3 (10/11, 17/11, 24/11) é que põe em prática o que foi desenhado aqui, até a defesa do produto funcionando na N3 (01/12). O código mostrado neste checkpoint é aproximado: pode e deve mudar quando for implementado de fato no ESP32.

Entrega em duas etapas, hoje — duas coisas em cada uma: o PDF no Teams e o repositório no GitHub.

Até 22h30 de hoje (20/10):

— PDF com as seções 1 a 4 preenchidas — ponto de partida, upgrade por camada, prova do quo a equipo leu o piso R1—R18 e já mapeou cada requisito para o próprio projeto, e a arquitetura-alvo esboçada (diagrama de blocos, fluxo do dado o fluxo do comando). Nenhuma dessas quatro seções depende de teoria técnica nova — usem o que já foi visto na Aula 11 e a leitura autônoma do piso.

— Repositório GitHub criado (se ainda não existir) ou atualizado, com um `README.md` simples: nomo do projeto, problema ou usuário, uma frase sobre o estado atual (herdado da N1)e para onde o upgrade está indo. Não precisa das instruções de instalação ainda — isso só é exigido no fim (R11, válido para a N3).

Até o final da Aula 13 (27/10/2026, 22h30): PDF completo, com as seções 5 a 9 (hardware calculado, firmware em fluxograma, contrato congelado v1, tecnologias, cronograma/riscos/divisão), o repositório atualizado de novo (seção 9). Essas seções dependem da teoria do hardware seguro, firmware-base e contratos, dada na própria Aula 13 — a oficina daquela noite é o tempo de aplicar o conteúdo ou fechar o documento.

Todos os integrantes devem revisar o documento inteiro antes do envio final: o conteúdo é objeto da prova individual da N2 (03/11). Esto checkpoint é a primeira (e única) entrega N2 — desenhe a documentação do projeto (grupo, 40% da N2); a entrega completa, na Aula 13, já congela o contrato como v1 (não há um CP13 separado). A N2 (03/11) — T_2 (40%) + P_2, prova individual de 6 questões (60%) — o conteúdo deste checkpoint é também o que cai na prova.

A partir de hoje, pensem o projeto como produto, não como exercício acadêmico (ver subseção "Como apresentar" em `requisitos projeto N3. pdf`): proposta do valor clara, cuidado visual e praticidade do uso. Isso não tem campo de pontuações próprio neste checkpoint; é a lente que vale desde já para a interface e o acabamento do hardware.

1. Ponto de partida: o projeto da N1

Parte da entrega parcial de hoje (22h30).

Problema e usuário	**Problema**: Plantas em vasos sofrem por irrigação incorreta (excesso ou falta d'água). Sem automação, o cuidador depende de rotinas manuais sujeitas a esquecimento. **Usuário/Contexto**: Pessoas que cultivam plantas em casa/escritório e desejam monitorar remotamente a umidade do solo e radiação/calor solar controlando a irrigação via web.
O que funcionou na N1	● Leitura em tempo real da umidade do solo. ● Publicação contínua de dados via MQTT com telemetria básica. ● Acionamento remoto da bomba d'água por comando da interface web. ● Reconexão automática ao Wi-Fi e Broker em caso de falha de rede.
Limitações conhecidas	Sensor só identifica umidade quando totalmente

	submerso em água. Não identifica umidade na terra
Proposta de valor (por que alguém usaria/compraria este produto)	O produto é útil para quem deseja monitorar o nível de umidade de suas plantas de forma a não regá-las excessivamente (sensor de umidade) ou esquecer de regá-las (modo automático)

Pedido de troca de projeto: ( ) Sim (X) Não

Se sim, justificar por que o projeto da N1 não comporta o upgrade o qual problema/usuário a equipe propõe no lugar. Só é válido com aprovação do professor.

2. Upgrade por camada

Parte da entrega parcial de hoje (22h30).

Para cada camada, descrever o estado na N1 e o que muda na N3.

Camada	Como era na N1	Como será na N3
Hardware	ESP + SENSOR + RELÉ + BOMBA + PROTOBOARD	ESP + SENSOR + RELÉ + BOMBA + PROTOBOARD + ESP + PROTOBOARD + SENSOR
Firmware	O SENSOR CAPTAVA A UMIDADE E MANDAVA PARA O BROKER, SE ACIMA DE UM CERTO NÍVEL DE UMIDADE A BOMBA LIGAVA	MANTÉM-SE A LÓGICA DE MEDIÇÃO DA UMIDADE/IRRIGAÇÃO, MAS SERÁ TAMBÉM IMPLEMENTADO UM SENSOR DE CALOR QUE ENVIARÁ UM ALERTA CASO A PLANTA DEIXE DE RECEBER LUZ SOLAR POR MUITOS DIAS.
Comunicação (MQTT)	ERA FEITA POR MEIO DE 4 TÓPICOS	SERÁ ACRESCENTADO UM TÓPICO PARA O SENSOR DE CALOR

Backend	Guarda dados de telemetria e alertas do sistema	Guarda dados de telemetria e alertas do sistema
Banco do dados	PostgreSQL hospedado no Supabase	PostgreSQL hospedado no Supabase
Interface	Botão para desligar/ligar bomba de irrigação, botão para alteração de modo do sistema (automático e manual) + dashboard com logs e monitoramento da umidade da planta	Botão para desligar/ligar bomba de irrigação, botão para alteração de modo do sistema (automático e manual) + dashboard com logs e monitoramento da umidade e medição de calor da planta
Decisão/regra	Bomba de irrigação: umidade acima de 60% desligava e abaixo de 30% ligava automático	Bomba de irrigação: umidade acima de 60% desliga e abaixo de 30% liga automático Bomba de calor: caso a planta não receba luz solar por mais de 3 dias, um alerta será enviado ao usuário (meio de comunicação a definir – e-mail, telegram ou semelhante).

3. Conferência contra o piso R1—R18

Parte da entrega parcial de hoje (22h30).

Para cada requisito do piso (ver `requisitos_projeto_N3. pdf`), uma frase sobre como o projeto pretende atendê-lo. Quem ainda não sabe, escrever a pendência — não deixar em branco. Vários requisitos do R12 a R18 já foram implementados por algumas equipes na N1: nesse caso, descrever o que já existe ou o que falta evoluir.

Req.	**Como o projeto vai atender**
R1	Além do sensor de umidade e bomba de irrigação, será implementado

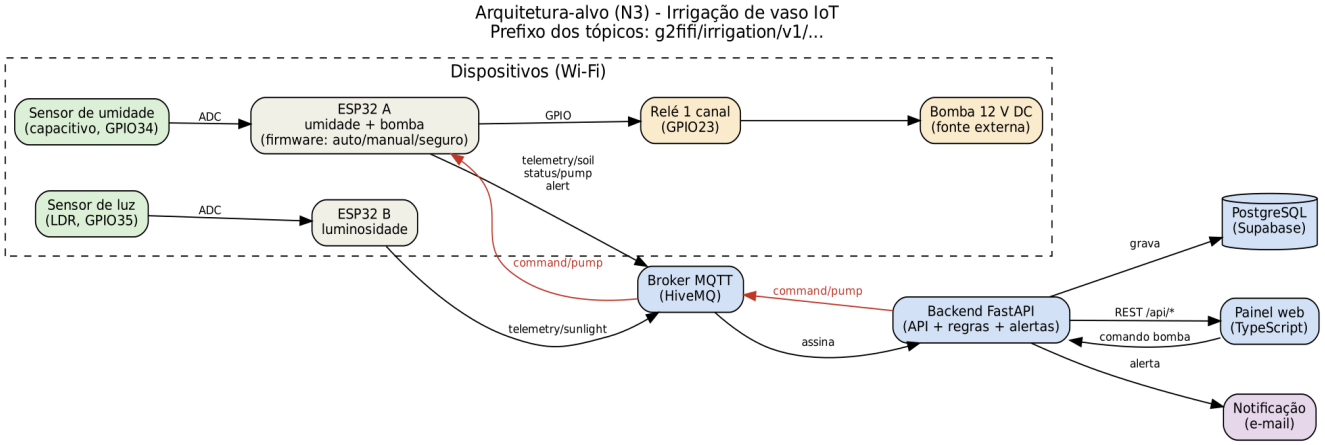
Hardware	um sensor de calor para saber se a planta pegou pouco sol nos últimos dias
R2 Firmware	Implementado: ● Leitura constante dos dados de umidade ● Validação da leitura ● Regra local (ativação automática da bomba a depender do nível de umidade do solo) ● Reconexão automática ● Timestamps e logs dos dados de umidade A implementar: ● Parametrização dos valores que ativam/desativam a bomba (alterados via front-end) ● Leitura constante dos dados de calor ● Timestamps e logs dos dados de calor
R3 Contrato MQTT	Implementado: ● CONTRATOS.md versionado na raiz do repositório, com a árvore de tópicos g2fifi/irrigation/v1/ (telemetry/soil, telemetry/sunlight, status/pump, alert, command/pump) e o formato JSON dos payloads.
R4 Backend	Implementado: ● API back-end com rotas relacionadas à umidade Pendente: ● Endpoints relacionados à leitura de calor ● Envio de alerta ao usuário em caso de pouca incidência de sol na planta ● Envio de valores para configuração da ativação/desativação automática da bomba ● Envio de quantidade de dias sem sol para receber alerta ● Documentação da API (Swagger ou postman) ● Sistema de login ● Segurança dos endpoints (exigir autenticação)

R5 Banco de dados	Implementado: ● Banco de dados PostgreSQL hospedado no Supabase Pendente: ● Criar tabelas relacionadas à medição de calor
R6 Interface	Implementado: ● Exibição da umidade do solo em tempo real ● Botão para ativar irrigação remota ● Botão para alterar modo (auto/manual) do sistema A implementar: ● Nova tela dedicada à exibição de logs por tipo (histórico de comandos, data e hora do último contato, estado dos dispositivos, alertas) ● Permitir alterar configuração de valor mínimo/máximo de umidade para ativação/desativação automática da bomba ● Permitir alterar tolerância de dias sem incidência de calor antes de enviar alerta ao usuário ● Tela de login
R7 Ciclo do comando	Sensor de Umidade → ESP32 → Firmware → Wi-Fi → MQTT Broker (HiveMQ) ↓ Painel Web (dashboard/) ↓ Comando → Bomba de irrigação Sensor de Calor → ESP32 → Firmware → Wi-Fi → MQTT Broker (HiveMQ) ↓ Painel Web (dashboard/)
R8 Decisão remota	Implementado: ● Permitir ativar/desativar a bomba através de botão na interface web

R9 Segurança básica	Implementado: credenciais fora do Git (back/.env e firmware/irrigacao/secrets.h, com modelos .env.example e secrets.h.example). Pendente: autenticação nos endpoints e senhas de usuários com hash no banco.
R10 Falhas testadas	• Tempo limite da bomba: Bomba desliga automaticamente após 30s de acionamento contínuo. • Queda de Wi-Fi: O ESP32 mantém a regra local no loop e reconecta em segundo plano sem travar. • Perda de sensor (NaN/Erro no ADC): O atuador assume estado seguro (desligado) e envia log de erro. • Falha no Broker MQTT: Tenta reconexão exponencial mantendo a operação offline de segurança.
R11 Entrega executável	No repositório, no read, está tudo documentado de como o projeto deve ser executado por qualquer um
R12 Orquestração (Docker Compose)	Será configurado um docker compose com todos os containers necessários
R13 Múltiplos dispositivos	2 sensores, um de umidade e um de calor
R14 Painel analítico	Terá um dashboard no projeto, pegando todos os dados enviados pelo sensor no período selecionado para a análise.
R15 Notificação externa	O backend (não o firmware) assina o tópico g2fifi/irrigation/v1/alert e envia e-mail ao usuário via SMTP configurável (SMTP_HOST, ALERT_EMAIL_TO); implementado, a validar na Unidade 3.
R16 Segurança avançada	Será aplicado TLS para criptografia das informações que trafegam em claro

(TLS/porfis)	
R17 OTA	O firmware será atualizado com base nas versões mais recentes nos sites oficiais
R18 Testes e CI	O backend terá código com CI-CD validando todos os testes de ponta a ponta

4. Arquitetura-alvo



Fluxo de dados (sensor → firmware → MQTT → backend → banco → interface):

1. O ESP32 responsável pelo sensor lê periodicamente a umidade do solo.
2. O firmware converte a leitura em porcentagem e publica a telemetria via MQTT.
3. O broker MQTT encaminha os dados ao backend.
4. O backend registra as leituras no PostgreSQL (Supabase).
5. A interface web consulta o backend e apresenta o histórico, o estado atual e o modo de operação.

Representação:
[Sensor de Umidade] → [ESP32 Sensor] → [MQTT] → [Backend] → [PostgreSQL/Supabase] → [Painel Web]

Fluxo de comando (interface → backend → MQTT → firmware → atuador → confirmação):

1. O usuário envia um comando pelo painel web (ligar, desligar ou alterar o modo).
2. O backend valida a solicitação e publica o comando no tópico MQTT correspondente.
3. O ESP32 responsável pela bomba recebe o comando.
4. O firmware aciona ou desliga o relé que controla a bomba.
5. O ESP32 publica um status de confirmação.

6. O backend atualiza o banco de dados e a interface exibe o novo estado.

Representação:
[Painel Web] → [Backend] → [MQTT] → [ESP32 Bomba] → [Relé] → [Bomba] → [Confirmação MQTT] → [Backend] → [Painel]

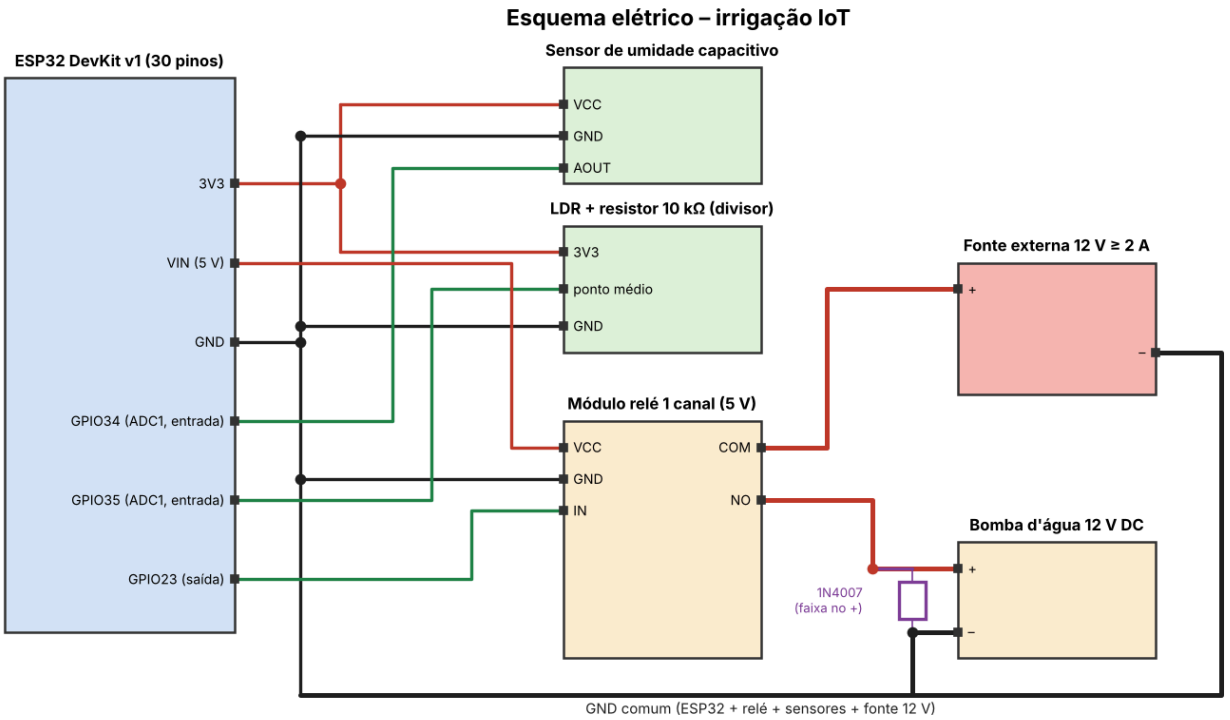
5. Hardware: desenho e dimensionamento

5.1 Verificação elétrica (calculada)

Sem bancada nesta etapa: os valores abaixo são calculados a partir do datasheet dos componentes escolhidos, não modificados. Anexar o esquema elétrico desenhado (à mão, Fritzing, KiCad ou equivalente) — não é preciso o circuito físico montado.

Verificação	Valor calculado/esperado	Como será garantido
Tensão no pino 3V3 e 5V/VIN usados	3.3V (ESP32 / Sensores) / 5V (Módulo Relé)	Regulador de tensão interno da placa ESP32 alimentado via USB 5V.
Corrente estimada do(s) atuador(es)	Bomba 12 V DC: fonte externa 12 V ≥ 2 A, dimensionada com margem	Módulo relé com optoacoplador; bomba 12 V alimentada por fonte externa dedicada, isolada do ESP32 pelo contato NO do relé.
GND comum entre todas as fontes previstas	0V	Barramento GND unificado na protoboard interligando ESP32, relé e fontes.
Maior tensão que chegará a um GPIO (qual sensor?)	3.3V	Sensores analógicos alimentados em 3.3V; pinos operando estritamente dentro de 0-3.3V.

Proteção planejada para carga indutiva (se houver)	Diodo Flyback (1N4007)	Diodo 1N4007 externo em antiparalelo com a bomba (o módulo relé protege apenas a própria bobina).
Estado seguro definido para o atuador no boot	Bomba DESLIGADA (LOW/HIGH conforme lógica do relé)	O firmware grava o nível de relé desligado (HIGH, relé ativo em nível baixo) no início do setup(), antes de conectar ao Wi-Fi.



Conferir no desenho:

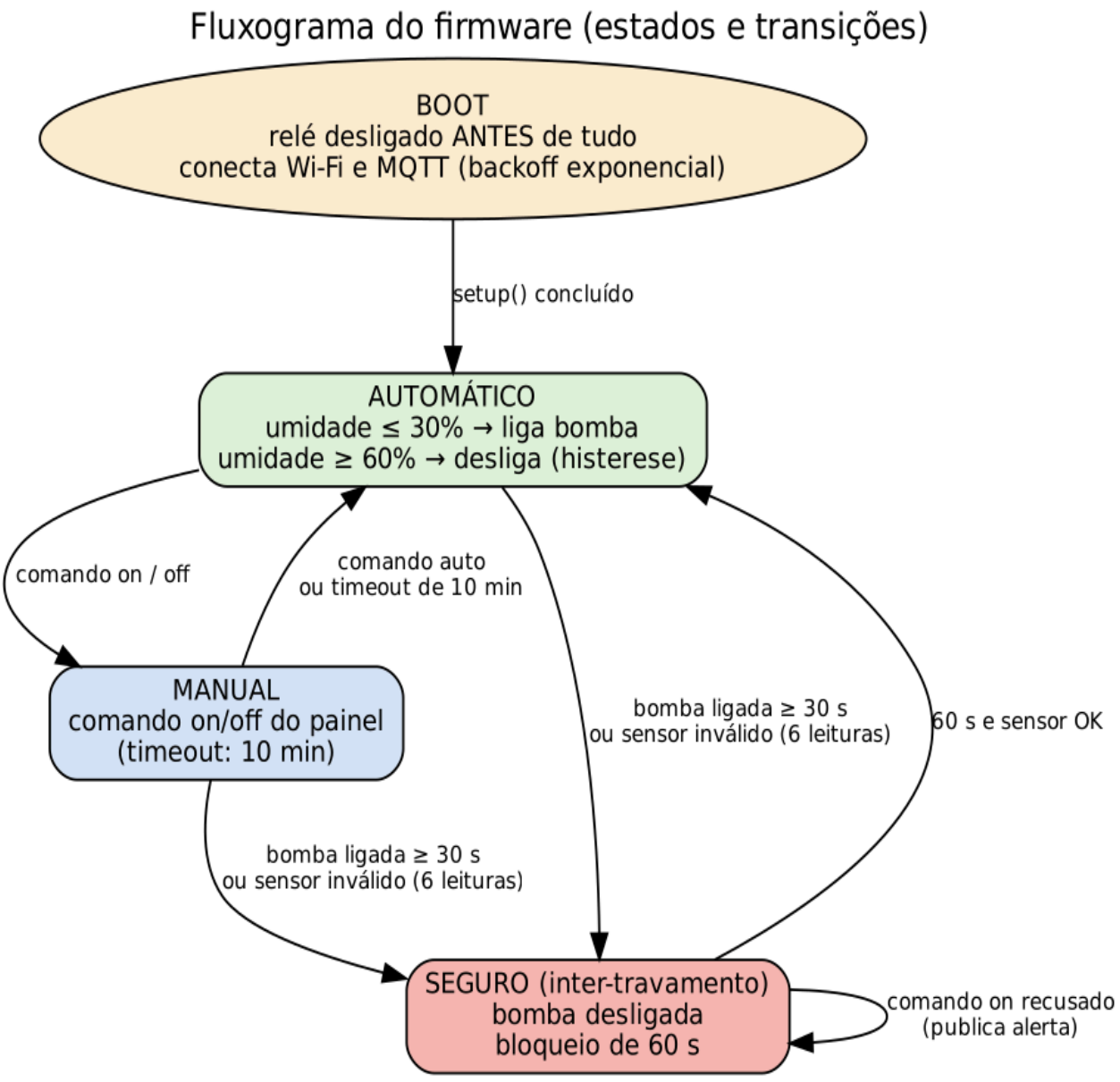
- ☐ Nenhuma carga (motor, bomba, rolé, coolor, fita de LED) alimentada diretamente por GPIO no esquema.
- ☐ Carga indutiva com diodo de roda livre previsto (ou módulo que já o inclua — indicar qual).

- □ Nenhum sinal do 5 V ligado direto a um GPIO no desenho (divisor ou conversor do nível quando necessário).
- □ Nenhum uso do GPIO 6—11; nenhum atuador em pino do £tFOppïDp (0, 2, 5, 12, 15) sem justificativa.
- □ Sensores analógicos planejados no ADC1 (GPIO 32—39), já que o ADC2 não lê com Wi-Fi ativo.

5.2 BOM e mapa de pinos

BOM — o que já tem	ESP32, Sensor de Umidade de Solo Capacitivo, Módulo Relé 1 Canal, Bomba de água 12 V DC, fonte externa 12 V ≥ 2 A, diodo 1N4007, Protoboard, Jumpers.
BOM — o que falta comprar/providenciar	Sensor de Luz/Luminosidade (LDR ou BH1750 para medição de calor/incidência solar).
Caminho do mapa de pi-nos no repositório	docs/hardware/pinout_map.md

5.3 Fluxograma do firmware



Desenhar os estados do firmware (pelo menos automático, manual e seguro), indicando em cada transição o evento que a provoca, o timeout do modo manual e a condição do inter-travamento. É um fluxograma do dosign: a implementação real no ESP32 pode ajustar detalhes.

6. Contrato v0

Na entrega parcial do hojo, ainda não precisa estar congelado. Na entrega completa (final da Aula 13), já deve estar congelado como v1. Escrever a árvore de tópicos pretendida e um exemplo do payload.

Árvore de tópicos	g2fifi/irrigation/v1/telemetry/soil (ESP32 → Broker): umidade do solo, estado da bomba e modo g2fifi/irrigation/v1/telemetry/sunlight (ESP32 → Broker): incidência de calor/luz g2fifi/irrigation/v1/command/pump (Backend → ESP32): comandos on, off, auto g2fifi/irrigation/v1/status/pump (ESP32 → Broker): confirmação do estado da bomba; g2fifi/irrigation/v1/alert (ESP32 → Broker): alertas de solo seco, segurança e sensor. Contrato completo em CONTRATOS.md (raiz do repositório)
Exemplo de payload (telemetria ou estado)	g2fifi/irrigation/v1/telemetry/soil: {“umidade”: 42, “raw”: 2310, “estado”: “off”, “modo”: “auto”} g2fifi/irrigation/v1/status/pump: {“bomba”: “on”, “modo”: “manual”, “origem”: “manual”, “online”: true} g2fifi/irrigation/v1/telemetry/sunlight: {"luminosidade": 78, "raw": 3200}; g2fifi/irrigation/v1/command/pump: texto simples “on”, “off” ou “auto”

7. Tecnologias pretendidas

Escolha livre da linguagem, framework, banco e interface: vedadas plataformas prontas e fechadas em que backend, regras, banco ou interface não possam ser inspecionados no ropositório (ex.: Blynk, Arduino IoT Cloud). Em caso de dúvida, perguntar antes de escolher.

Camada	Tecnologia escolhida	Juustificativa
Backend	python	Simplicidade da linguagem
Banco de dados	PostgreSQL (Supabase)	Hospedagem gratuita e interface simples de usar
Interface	TypeScript	Familiaridade de uma das membros da equipe com o Framework
Broker MQTT	HiveMQ	Broker WEB simples de utilizar
Orquestração (Docker Compose)	Docker Compose (containers: back FastAPI e front)	Sobe backend e painel web com um único comando (docker compose up), facilitando a execução em qualquer máquina

8. Cronograma, riscos e divisão de responsabilidades

8.1 Cronograma até a N3

Data	Marco	O que a equipe entrega
27/10	CP12 completo (fim da Aula 13) + contrato congelado v1	PDF completo e repositório atualizado: README, contrato MQTT v1 congelado, documentação de hardware e arquitetura
03/11	N2	Documentação do projeto (T2) entregue e prova individual P2 sobre o conteúdo deste checkpoint
10/11 — 24/11	Unidade 3	Implementação: firmware com sensor de luz e regras de falha, backend com login e novos

		endpoints, front com login, logs e configurações, Docker Compose, TLS, testes e CI
01/12	N3	Defesa do produto funcionando, com README de execução completo (R11)

8.2 Riscos

Risco	Mitigação
Corrosão rápida do sensor resistivo pelo contato constante com o solo	Uso de sensor capacitivo e leitura periódica espaçada
Queda do Wi-Fi ou do broker MQTT	Reconexão não bloqueante no ESP32, mantendo a regra local da bomba ativa mesmo offline
Bomba ligada por tempo excessivo ou funcionando a seco	Trava de segurança no firmware: desliga após 30 s de acionamento contínuo e bloqueia novo acionamento automático por 60 s
Broker HiveMQ público e aberto: outro grupo pode publicar nos mesmos tópicos	Prefixo exclusivo do grupo nos tópicos e, no N3, TLS e autenticação no broker (R16)

8.3 Divisão de responsabilidades

Integrante	Responsabilidade principal atU a entrega completa (Aula 13)
Ana Julia Castelo Branco	A definir
Graziela Torres	Front-end: painel web em TypeScript (inicialização e estrutura do projeto)
Henrique Xavier Klappoth	Simulação no Wokwi, firmware inicial e painel web MQTT
Marina Rosa Oliveira	Backend FastAPI, modelagem do banco (diagrama

	MER) e documentação no README
Rodrigo Xavier Klappoth	Correções do backend e documentação de hardware (esquema de conexões e pinos)
Sophia Eggert	Integração backend e firmware, configuração do ambiente e organização do README

9. Repositório (GitHub) — 1,0 ponto fixo (25% do T_2)

O repositório é a segunda entrega de hoje, junto com este PDF (ver Orientações). Precisa existir com um `README.md` simples — nome do projeto, problema ou usuário, estado atual (herdado da N1) e para onde o upgrade está indo — desde às 22h30. Não é o README do instalação completo: isso só é exigido para a N3 (R11).

A partir de 20/10/2026, a completude do repositório tem nota própria e fixa dentro do T_2: 1,0 ponto dos 4,0, avaliado na entrega completa (final da Aula 13). O princípio é simples: o PDF é a proposta, o repositório é a prova — o que está escrito aqui precisa existir, de fato, versionado no GitHub.

Critério de avaliação:

Completo (1,0 ponto) — todos os itens presentes: repositório público (acessível sem convite); `README.md` comploto (projeto, probloma, usuário, estado atual, direção do upgrade); diagramas e gráficos versionados no repositório (arquitetura-alvo, fluxograma do firmware, esquema elétrico — não apenas texto); conteúdo do repositório correspondendo exatamente ao que foi entregue em PDF (mesmo commit, sem divergência); commit de cada integrante da equipe.

Parcial (0,5 ponto) — falta um item secundário: por exemplo, só texto sem diagramas versionados, ou commits concentrados em 1—2 integrantes.

Insuficiente (0 ponto) — repositório não existe, não é público, ou diverge do PDF entregue.

Link do repositório	https://github.com/fifieggert/iot_irrigation
Hash do commit entregue	65cb920c4e9df6896ed024c4eea6480420f5f2e4

Integrante	Usuário GitHub (com commit próprio no repositório)
Ana Julia Castelo Branco	Pendente (ainda sem commit)
Graziela Torres	grazielatrres
Henrique Xavier Klappoth	Ikeew
Marina Rosa Oliveira	mari-r0sa
Rodrigo Xavier Klappoth	RodrigoKlappoth
Sophia Eggert	fifieggert

Checklist antes do envio

- □ Seções 1 a 4 enviadas até 22h30 do hoje (mesmo que o resto ainda esteja em branco).
- □ Repositório no GitHub criado/atualizado até 22h30, com `README.md` simples apontando para onde o projeto está indo. Repositório público — o ponto do GitHub (seção 9) exige acesso sem convite.
- □ Tabela de verificação elétrica preenchida com valores calculados, não medidos — sem bancada nesta etapa.
- □ Esquema elétrico desenhado anexado (não é necessário circuito físico montado).
- □ Fluxograma do firmware desenhado.
- □ Conferência R1—R18 sem linha om branco (pendências declaradas, não omitidas).
- □ Cronograma até a N3 preenchido.
- □ Tecnologias protendidas justificadas; nenhuma plataforma fechada da seção 4.1.
- □ Repositório com commit de cada integrante listado.
- □ Todos os integrantes revisaram o documento completo antes do envio final.

Entrega

Duas etapas, cada uma com duas coisas — PDF no Teams e repositório no GitHub:

1. Até 22h30 de hoje (20/10): PDF com polo monos as soçõos 1 o 3 (nome sugerido: `CP12_EQUIPE—X_PARCIAL. pdf`) e ropositório criado/atualizado com `README.md` simples (projeto, problema, usuário, estado atual, direção do upgrade).
2. Até o final da Aula 13 (27/10/2026, 22h30): PDF completo, com todas as seções (nome sugerido: `CP12_EQUIPE—X_PRDJETD_V1. 0. pdf`) e repositório atualizado do novo, com o contrato já congelado como v1.